

Lab. 6
Full-Stack CRUD Application

Content

1. Separation of concerns
2. A connection pool
3. Routers: grouping related routes
4. Controllers: separating the logic
5. Validating input and returning 400 for bad requests
6. Completing CRUD over HTTP: PUT and DELETE
7. A fuller front-end: list, add and delete with fetch()
8. Where we're going next (Lab 7)
9. Summary

Recap

- In Lab 5 every route, every query and the connection all lived in one app.js.
- That is fine for a demo, but it grows hard to read, test and change as routes pile up.
- Today we keep exactly the same behavior, but organize it into clear, reusable parts.
- We also finish CRUD over HTTP and give the API a proper front-end page.

Separation of Concerns

- Good structure splits a program by job, so each part does one thing well.
- Database setup (the pool): one module that everything shares.
- Routes: which URL plus method maps to which handler.
- Controllers: the actual logic: read the request, validate, query, respond.
- Smaller files are easier to read, reuse and fix, and easier to grade.

A Connection Pool

Open many connections once and reuse them, instead of creating a new one on every request.

```
// db.js
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost', user: 'root',
  password: 'your_password', database: 'university',
  connectionLimit: 10
});

module.exports = pool;
```

A pool hands out ready connections and takes them back automatically, faster and simpler than connecting each time.

A Connection Pool (Contd.)

Using the Pool: Import the pool anywhere and query it directly. No createConnection, no manual end().

```
const pool = require('./db');

const [rows] = await pool.query('SELECT * FROM students');

const [r] = await pool.execute(
  'SELECT * FROM students WHERE id = ?', [id]);
```

pool.query() and pool.execute() work just like before; the pool manages the connections for you.

Routers: Grouping Related Routes

An express.Router() is a mini-app: define related routes in their own file, then mount it.

```
// routes/students.routes.js
const router = require('express').Router();
const c = require('../controllers/students.controller');

router.get('/',     c.list);
router.get('/:id',  c.getOne);
router.post('/',    c.create);
module.exports = router;

// app.js:  app.use('/students', studentsRouter);
```

Each path here is relative to the mount point, so router.get('/') answers GET /students.

Controllers: Separating the Logic

A controller is a function (req, res) that does the work. Routes just point to them.

```
// controllers/students.controller.js
const pool = require('../db');

exports.list = async (req, res) => {
  try {
    const [rows] = await pool.query('SELECT * FROM students');
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

The route says 'when GET / happens, run list'; the controller holds the how. Test and reuse each on its own.

Controllers: Separating the Logic (Contd.)

File structure so far & beyond: A small, predictable layout. Each file has one job, and app.js just wires them together.

```
project/
  app.js                 # create app, mount routers, listen (later: Ex. 1)
  db.js                  # the shared connection pool (done)
  routes/
    students.routes.js   # paths -> controller functions (done)
  controllers/
    students.controller.js  # the logic (done) + validation (later)
  public/
    index.html  main.js  styles.css   # the front-end (later)
```

When you add a feature, you know exactly where each piece goes.

Controllers: Separating the Logic (Contd.)

Lab Exercise 1: ***Refactor into structure***

- ❑ Create db.js that exports a mysql2 pool, and import it where you query.
- ❑ Move your student routes into routes/students.routes.js and mount them in app.js.
- ❑ Move the route logic into controllers/students.controller.js.
- ❑ Confirm GET /students still works exactly as before.

Starter code:

```
// app.js
const studentsRouter = require('./routes/students.routes');
app.use('/students', studentsRouter);
```

Test command for ex1. INSERT

- curl -X POST http://localhost:3000/students -H "Content-Type: application/json" -d '{"name":"Daniel","year":3,"major":"Data Science"}'

Validating input and returning 400 for bad requests

Don't Trust the Client

- Anything sent to your API might be wrong, missing, or malicious: check it before you use it.
- Validate that required fields are present and have the right type and a sensible range.
- If the input is bad, stop early and reply 400 Bad Request with a helpful message.
- Validation protects your data; the ? placeholder protects against injection, you want both.

Validating input and returning 400 for bad requests (Contd.)

Validating in a Controller: Check the body first; only query the database once the input is known to be good.

```
exports.create = async (req, res) => {
  const { name, major, year } = req.body;
  if (!name || !major)
    return res.status(400).json({ error: 'name and major are required' });
  if (!Number.isInteger(year) || year < 1 || year > 4)
    return res.status(400).json({ error: 'year must be 1 to 4' });
  // ... insert with ? placeholders, then res.status(201) ...
};
```

Return early on the first problem. Libraries like express-validator can do this at scale, but the idea is the same.

Completing CRUD over HTTP: PUT and DELETE

Add the last two operations so the whole of CRUD is reachable over HTTP.

```
// PUT /students/:id  -- update
exports.update = async (req, res) => {
  const [r] = await pool.execute(
    'UPDATE students SET major = ? WHERE id = ?',
    [req.body.major, req.params.id]);
  if (r.affectedRows === 0) return res.status(404).json({ error: 'not
found' });
  res.json({ updated: r.affectedRows });
};
// DELETE /students/:id -> pool.execute('DELETE ... WHERE id = ?', [id])
```

****PUT updates, DELETE removes; both use ? and check affectedRows to return 404 when nothing matched. ***** see Lab Sheet for the complete code of DELETE ******

Completing CRUD over HTTP: PUT and DELETE (Contd.)

Lab Exercise 2: ***Validate and finish CRUD***

- ❑ Add validation to your create controller: reject bad input with 400 and a message.
- ❑ Add a PUT /students/:id to route + controller that updates a student.
- ❑ Add a DELETE /students/:id to route + controller that removes a student.
- ❑ Return 404 from PUT and DELETE when the id doesn't exist.

Starter code:

```
// students.routes.js
router.put('/:id',    c.update);
router.delete('/:id', c.remove);
```

Test command for ex2. UPDATE & DELETE

- curl -X PUT http://localhost:3000/students/1 -H "Content-Type: application/json" -d '{"major":"Math"}'
- curl -X DELETE http://localhost:3000/students/1

*** Update student with ID=1 and Delete student with ID=1. You might change the ID for testing.

A fuller front-end: list, add and delete with fetch()

- ✓ Now build a page that really uses the API, not just a read-only list.
- ✓ It will list all records, add a new one through a form, and delete one with a button.
- ✓ Plain HTML, CSS and JavaScript talk to your API with fetch(): no extra framework needed.
- ✓ Each action calls the API, then re-loads the list so the page always shows the truth in the database.

A fuller front-end: list, add and delete with fetch() (Contd.)

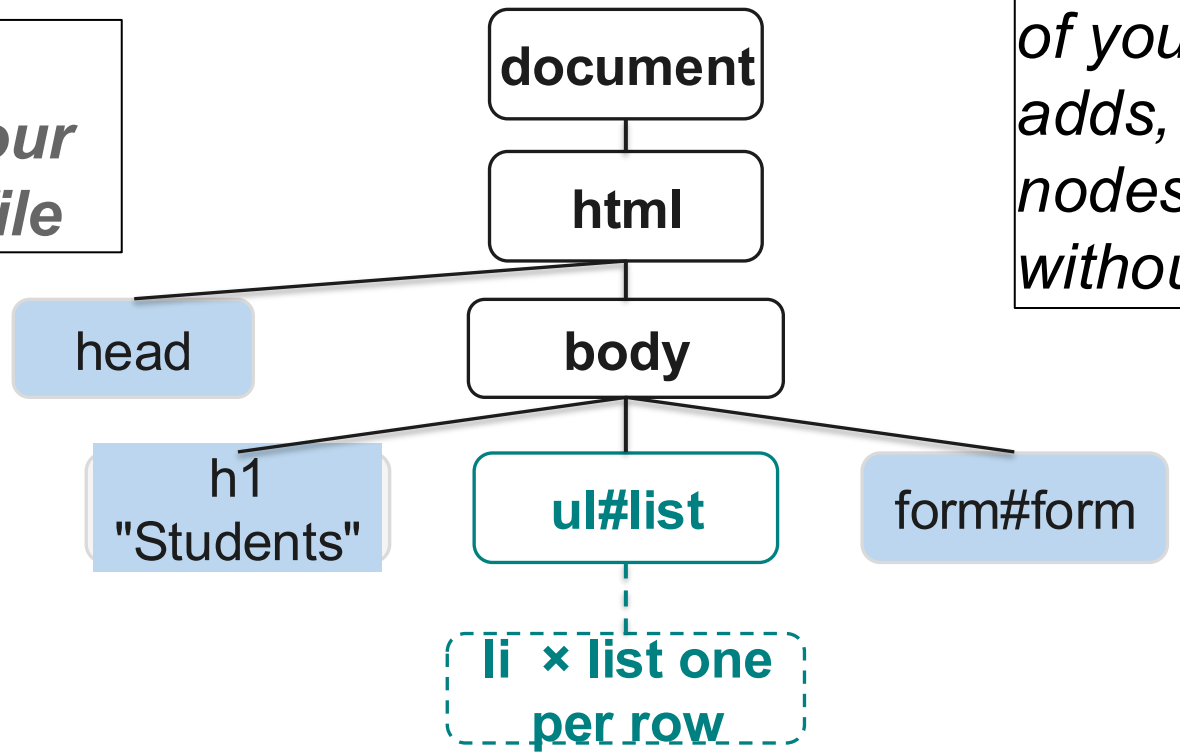
Front-End Structure: HTML, CSS, JS

- The same separation of concerns applies once you leave the server: three files, three jobs.
- HTML (index.html), the structure: what elements exist on the page.
- CSS (style.css), the presentation: colors, spacing, fonts, layout.
- JavaScript (main.js), the behavior: calls the API and edits the page's DOM (next slide) so it matches the data.
- Change how something looks without touching how it works, and vice versa.

A fuller front-end: list, add and delete with fetch() (Contd.)

The DOM: A Live Tree of Elements

Static from your HTML file



The DOM starts as a mirror of your HTML, then main.js adds, removes, or edits nodes, the page updates without a reload.

Dynamic: main.js creates one of these per row after fetch()

A fuller front-end: list, add and delete with fetch() (Contd.)

The HTML Skeleton: Structure first, a container for the list, and a form to add a new record.

```
<!-- public/index.html -->
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Students</h1>
  <ul id="list"></ul>
  <form id="form">
    <input name="name" placeholder="Name">
    <input name="major" placeholder="Major">
    <button type="submit">Add</button>
  </form>
  <script src="main.js"></script>
</body>
</html>
```

Every element JavaScript will touch needs an id, #list and #form are the hooks main.js looks for.

A fuller front-end: list, add and delete with fetch() (Contd.)

Styling with CSS: Presentation lives separately: link it once, then select elements by tag, class, or id.

```
/* public/style.css */
body {
  font-family: Arial, sans-serif;
  max-width: 480px;
  margin: 40px auto;
}

#list li {
  padding: 8px;
  border-bottom: 1px solid #ddd;
}

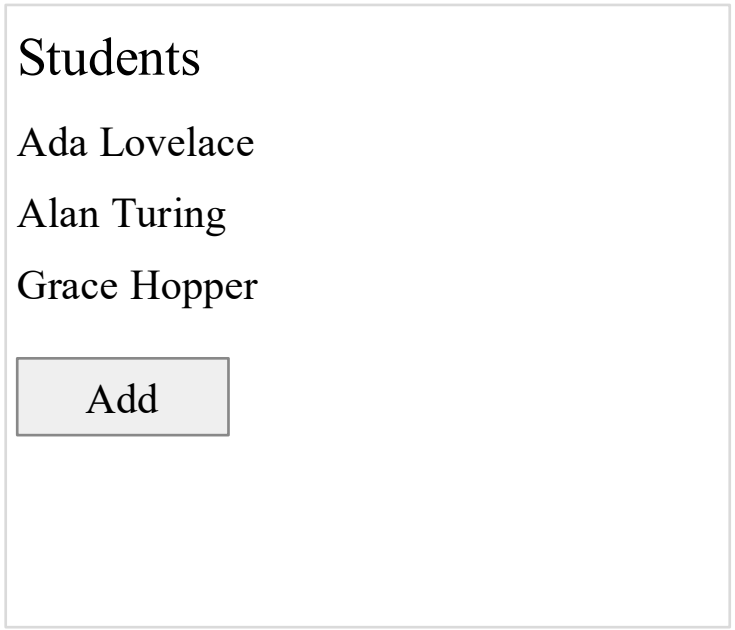
button {
  background: #1E8A82;
  color: white;
  border: none;
}
```

Selectors target the same ids the HTML defines #list, button: so structure and style stay linked but separate.

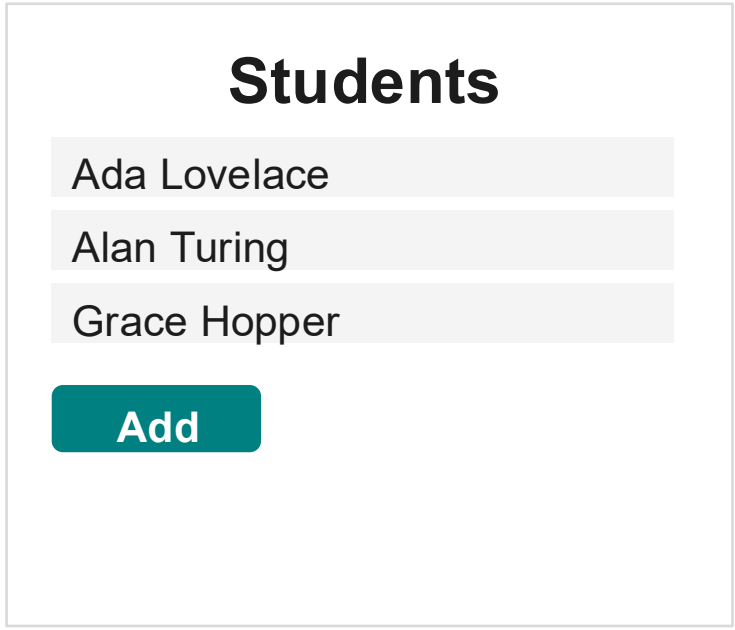
A fuller front-end: list, add and delete with fetch() (Contd.)

Before & After: Adding CSS

Before: HTML only



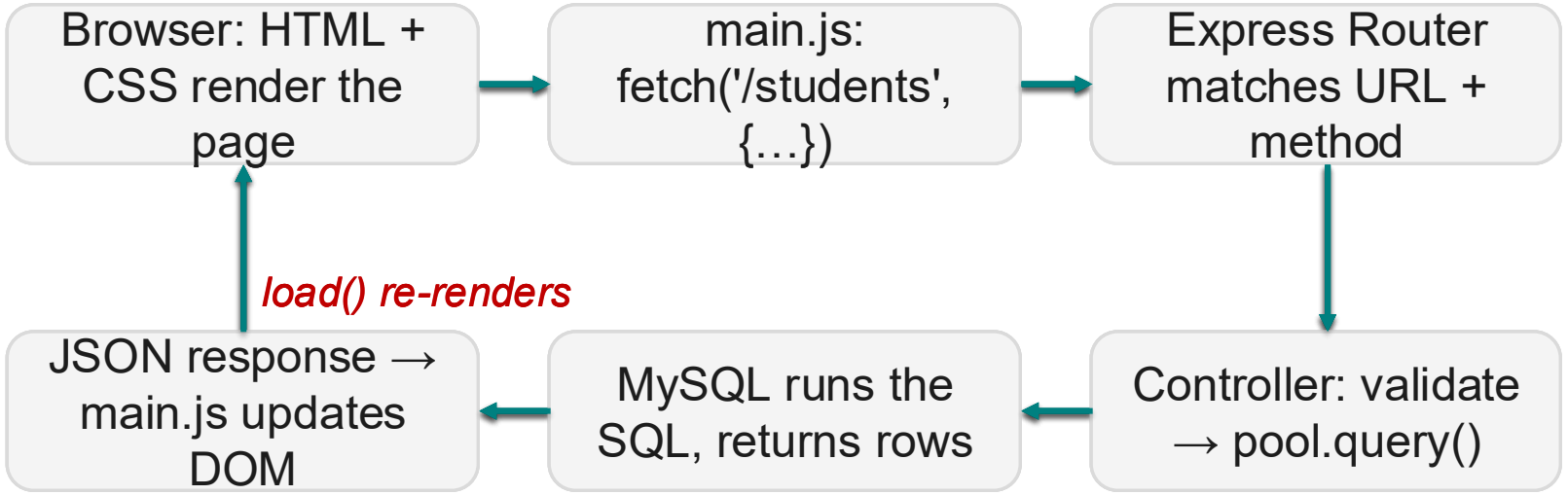
After: HTML + CSS



Same markup, different CSS, the HTML elements never change, only how they're painted.

A fuller front-end: list, add and delete with fetch() (Contd.)

How It All Connects: Request Flow



One request/response cycle: main.js repeats this every time the data changes.

A fuller front-end: list, add and delete with fetch() (Contd.)

Folder Structure with a Front-End: The public/ folder holds everything the browser loads directly, no build step needed.

```
project/
  app.js                # create app, mount routers, listen
  db.js                 # the shared connection pool
  routes/
    students.routes.js   # paths -> controller functions
  controllers/
    students.controller.js  # the logic + validation
  public/
    index.html  main.js  styles.css   # the front-end
```

Nothing in public/ needs a bundler or compiler, the browser loads these files exactly as written.

A fuller front-end: list, add and delete with fetch() (Contd.)

Putting HTML + CSS + JS Together

- index.html defines the elements; style.css decides how they look; main.js decides what they do.
- main.js never touches the database directly, it only talks to your Express API with fetch().
- Every action (add, delete) follows the same round trip: fetch → server → database → JSON → DOM.
- This is the same client/server split you'll see again in mobile apps and native software.
- Next: Lab Exercise 3 asks you to build exactly this page for your own resource.

A fuller front-end: list, add and delete with fetch() (Contd.)

Rendering the List with fetch: Fetch the data, then build the page from it. Re-run this whenever the data changes.

```
// public/main.js
async function load() {
  const rows = await (await fetch('/students')).json();
  list.innerHTML = rows.map(s =>
    `<li>${s.name} - ${s.major}
      <button data-id="${s.id}">delete</button></li>`
  ).join('');
}
load();
```

One load() function rebuilds the list from the API. Call it on start, and again after every change.

A fuller front-end: list, add and delete with fetch() (Contd.)

Adding and Deleting from the Page: Send the form as a POST, and the delete button as a DELETE; reload the list afterwards.

```
form.addEventListener('submit', async (e) => {
  e.preventDefault();
  await fetch('/students', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name: nameInput.value, major: majorInput.value,
year: 1 })
  });
  load();   // refresh the list
});
```

fetch's second argument sets the method, headers and JSON body. After the request, call load() again.

A fuller front-end: list, add and delete with fetch() (Contd.)

Lab Exercise 3: ***Build the front-end***

- ❑ In public/, make index.html with a list area and an 'add' form.
- ❑ In main.js, load() the list from your API and render it.
- ❑ Submit the form with a POST, then call load() to refresh.
- ❑ Add a delete button per row that sends DELETE and refreshes the list.

Starter code:

```
await fetch('/students/' + id, { method: 'DELETE' });
load();
```

A fuller front-end: list, add and delete with fetch() (Contd.)

Good Habits

- One shared pool, imported everywhere, never createConnection inside a route.
- Routes describe endpoints; controllers hold logic; db.js holds configuration. Keep them apart.
- Validate every incoming body and parameter; reply 400 on bad input, 404 on missing records.
- After any change on the front-end, re-load from the API so the page matches the database.

Where we're going next (Lab 7)

- You now have a structured web application: a pool, routers, controllers, validation and a working UI.
- In Lab 7 we build on this structure with the next topic in the course.
- This is the architecture your course project will use, keep your code organized this way from now on.

Summary

- Split the app by job: db.js (pool), routes/ (endpoints), controllers/ (logic), public/ (front-end).
- A pool reuses connections; import it and call pool.query / pool.execute.
- Validate input and return 400; complete CRUD with PUT and DELETE, returning 404 when missing.
- A plain HTML/JS page lists, adds and deletes through fetch(), reloading after each change.
- Complete Exercises 1–3 in VS Code during class, refactoring your Lab 5 API.

Thank you & Assignment time…